Reproducing a Hybrid Transformer Model for Wind Time Series Prediction

Group 2

DATA6550

Introduction

The current project demonstrates the findings from the research article which presents "A hybrid deep learning methodology for wind power forecasting based on attention" that Yıldırım Akbal and Kamil Demirberk Ünlü created. The researchers developed a compact hybrid deep learning model which uses convolutional networks and attention mechanisms together with a recurrent head for wind power prediction. The authors contend that their system design enables better management of limited data situations than transformers which use only deeper architectures to process data while still maintaining essential time-based patterns. The researchers achieved exceptional short-term results which reached an average R^2 score of approximately 0.99 but they observed that long-term predictions became increasingly challenging after 48 hours.

The project aimed to reproduce the study by creating all necessary model components, constructing a reproducible dataset, executing the forecasting process, and evaluating results against those documented in the original research. The paper employed the Global Energy Forecasting Competition 2012 Wind Forecasting dataset as its primary dataset which the researchers selected for their study. The dataset comprises wind power generation records from seven different wind farms along with their associated forecast-related documentation. The training file contains 18757 rows whereas the test file comprises 7488 rows and each of the seven forecast files contains 104832 rows.

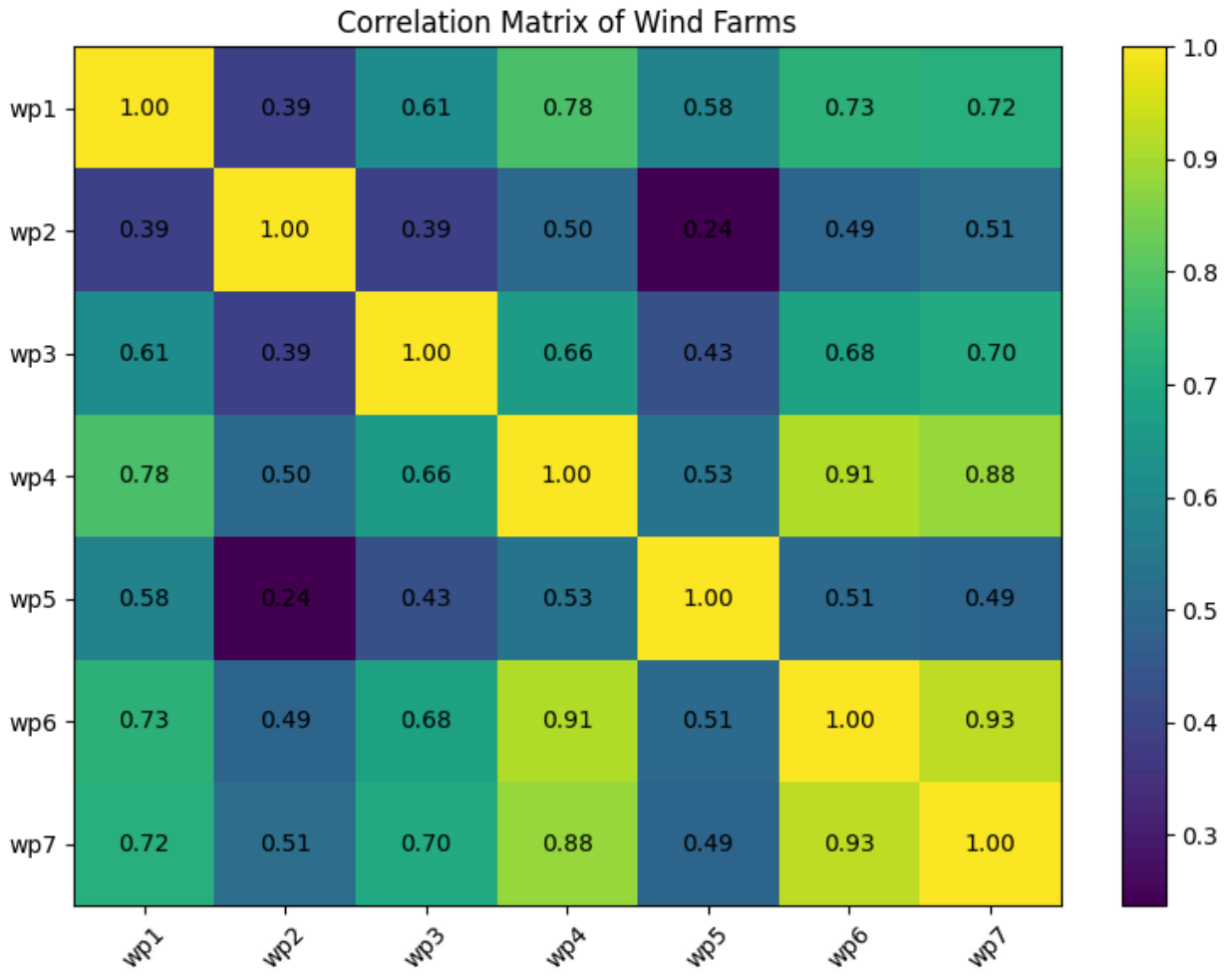
Before conducting model reproduction scientists investigated the dataset to establish its structural properties and its data quality. The target wind power variables (wp1 to wp7) are standardized to fall within a range that begins at 0 and ends at 1. The date columns converted to datetime format show that the training period spans from 2009-07-01 to 2012-06-26 while the test set extends to 2012-06-28. The analysis of missing values showed that the primary target files were complete but each forecast file contained 11160 missing values across the u v ws and wd parameters. The reproduction attempt required preprocessing work because it became a vital process for successful reproduction work.

The correlation analysis of seven wind farms demonstrated that three of the farms shared strong connections with each other. The strongest correlation between variables occurred between wp6 and wp7 (0.928) while the connection between wp2 and wp5 (0.238) showed a weaker relationship. The dataset contains shared structural elements together with unique farm-specific variations which make it appropriate for modeling through the hybrid architectural system designed in this research.

Methodology

Reproduction approach

The original implementation existed as a TensorFlow/Keras project which used script files to contain its helper functions and layers and training setup and main model. The developers transformed the code into Jupyter/Colab-compatible format to achieve notebook workflow reproducibility which they subsequently debugged through incremental steps. The paper establishes the reproduction framework the model uses through its first stage which employs convolution-based compression and its second stage which uses attention-based transformation blocks and its third stage which implements an LSTM forecasting head. The model functions as a univariate system according to the paper which uses a 512 lag size to enable learning from multiple wind farms through dedicated tokens for each time series.



The reproduction used the forecast files containing the ws variable because the public code is written to search for CSV files with a ws column and turn them into sliding windows. The system

successfully processed all seven forecast files which resulted in a total of 733824 wind speed observations from the seven farms. The researchers converted these sequences into supervised learning samples through a sliding window method which used a length of 512 to create input samples that included the first 512 values while the target value represented the subsequent value within the sequence.

Environmental setup

The notebook environment executed the work through Python programming together with TensorFlow/Keras and the NumPy and pandas and matplotlib and scikit-learn libraries. The original code required multiple modifications because it had been developed for command-line execution. The system required multiple changes because it needed to handle notebook environment system from command-line execution which did not use notebook variables.

The system needed multiple TensorFlow/Keras API problems to be addressed. The training flag needed to be passed in a way that created problems because the system had been designed to use specific layer arguments. The document states that the original model implemented the Adam optimizer with an initial learning rate of 0.0001 and used exponential decay after the fifth epoch and employed a dropout rate of 0.1 and utilized mean square loss for approximately 15 epochs with early stopping. The reproduced workflow used the training setup which employed Adam and mean squared error with a batch size of 64 and 15 epochs in a notebook environment. The researchers maintained most of the original study design while they could not confirm every single aspect of the training environment.

Data processing steps

The reproduction process relied heavily on data preprocessing activities. The paper reports the following preprocessing choices: missing values were imputed using cubic spline interpolation; maximum wind plant capacity determined the spikes which needed to be clipped; the time series data used mean and standard deviation for normalization; a 4:1 train-test split was used; and each time series was assigned an identifier between 0 and 6.

The wind speed series from the reproduced version underwent interpolation and normalization processes before the time series split into training and testing sections which followed an 80/20 distribution for each sequence. The system generated sliding windows which received class labels to show which wind farm each window belonged to. The input arrays were transformed into Conv1D, which required input in a specific format consisting of three dimensions that included batch, steps, and channels. The hybrid model used this method to correctly handle sequence data during its training sessions.

Results

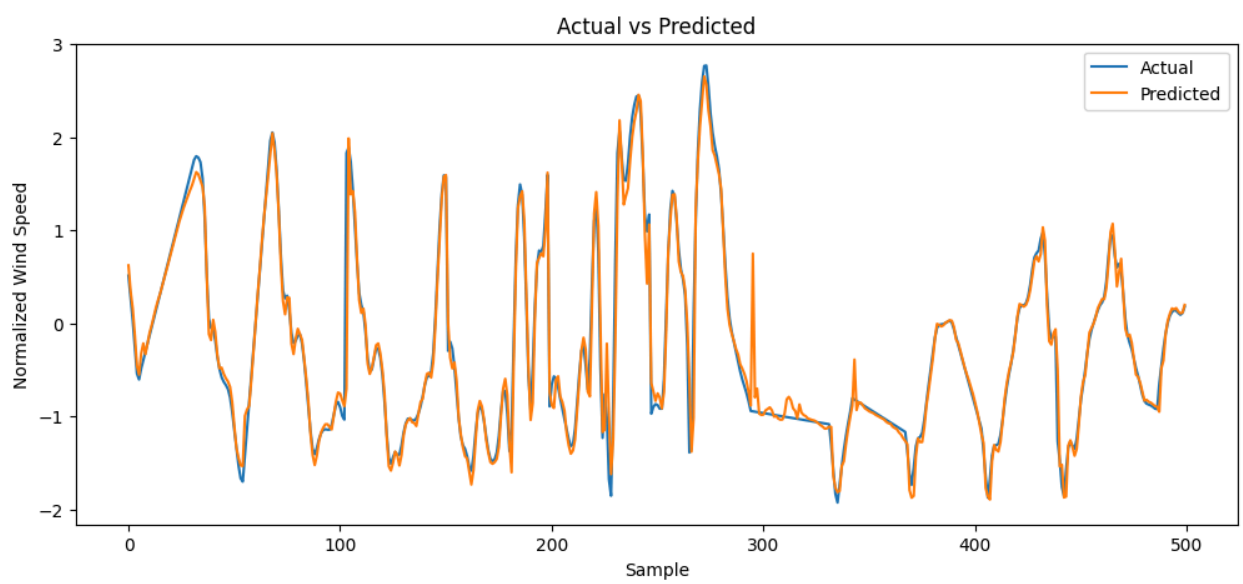
Comparison with original

The original paper reports that the largest model achieved an average $R^2 = 0.9930$ on the test set. The smallest model performance results in the paper show per-series R^2 values which range from 0.9916 to 0.9936 together with MSE values which extend from 0.000763 to 0.00103 and MAE values which range from 0.0162 to 0.0250.

In the reproduced implementation, the obtained results were:

- MSE = 0.0228
- MAE = 0.0730
- $R^2 = 0.9775$

The reproduced results demonstrate exceptional strength because they show that the model acquired valuable knowledge about time-based patterns. The training process showed continuous improvement through decreasing loss curves while the actual-versus-predicted plot displayed accurate sequence tracking. The reproduced numbers showed weaker results than the values presented in the original research document.



Discrepancy analysis

The reproduction achieved partial success because it produced partial results but failed to deliver complete success. The study confirmed strong short-term forecasting accuracy, but the study results could not attain the exact numerical results. The explanation for this difference exists because multiple causes create the situation. The path of the reproduced code depended

on the public implementation which used a ws-based workflow but the paper focused on wind power forecasting through seven wind farm time series.

The paper describes its preprocessing process with details about cubic spline interpolation and spike clipping because even small modifications to preprocessing procedures will lead to significant changes in final metric results. The notebook-based environment required multiple code fixes due to version and compatibility issues, which means the reproduced run was not a perfectly identical software environment.

The paper demonstrates training changes produced results that the authors could not reproduce, which indicates the authors discovered numerical results that changed when they used different conditions. The reproduced model achieved an R^2 value of 0.9775, which confirms the paper's primary assertion that the hybrid system successfully predicts wind-related forecasting tasks. The system obtained high R^2 results through the testing process.

Tuning Window Size

In this project, we have worked in groups to experiment on different window sizes to determine the amount of information required by the model for better predictions. Window size is the number of past time steps used by the model for prediction. Thus, choosing the right window size is extremely important. In this experiment, we have used different window sizes like 64, 128, 256, and 512 while keeping other parameters like the learning rate, batch size, and number of epochs fixed. After training the model on different window sizes, we have checked the results by using metrics like MSE, MAE, R^2 score, and validation loss. Based on our observations, we have concluded that window size 128 performs better than other window sizes. This is because we have obtained the lowest error values and the highest R^2 score, which means our model has performed better by getting predictions closest to actual values. While working on window size 64, we have seen that the model was not getting enough information about the past, resulting in poor performance. On the other hand, while working on window sizes like 256 and 512, we have seen slight degradation in performance because the model was getting too much information, including irrelevant information like noise. Thus, we have concluded that the model requires a balance between too little information and too much information. Based on the results obtained, we have chosen window size 128 as the best window size for further experiments because we have obtained better results compared to other window sizes.

Tuning Batch Size and Epochs

Once we have finalized our window size, as a group, we have worked together to tune our batch size and epochs in order to improve the learning of our model. The batch size is defined as the number of instances used for updation of our model, while the number of epochs represents the number of times our model has passed over our dataset. In order to perform this step, we have used a window size of 128, a batch size of 64, 2 epochs, and a constant learning rate of 0.001. Once our model has learned from our data, we have used MSE, MAE, and R^2 score for evaluating our model's performance. In addition to these, we have used a baseline model for evaluating our model's performance as compared to the baseline model. As per our results, it is clearly evident that our hybrid model has performed better as compared to our baseline model, as our error rate is less and our R^2 score is high. This clearly indicates that our model is learning efficiently from our data instead of sticking to our previous data. In addition to these results, our validation loss has remained constant, which clearly indicates that our model has not faced the problem of overfitting during our training phase. Although we have used only one combination for our batch size and epochs for our model in this notebook, it is clearly evident that our model has performed better for our batch size of 64 in terms of efficiency as well as our model's performance. In addition to these results, our model has performed better for 2 epochs instead of increasing our epochs, as increasing our epochs may cause our model to face the problem of overfitting.

Learning Rate Optimization

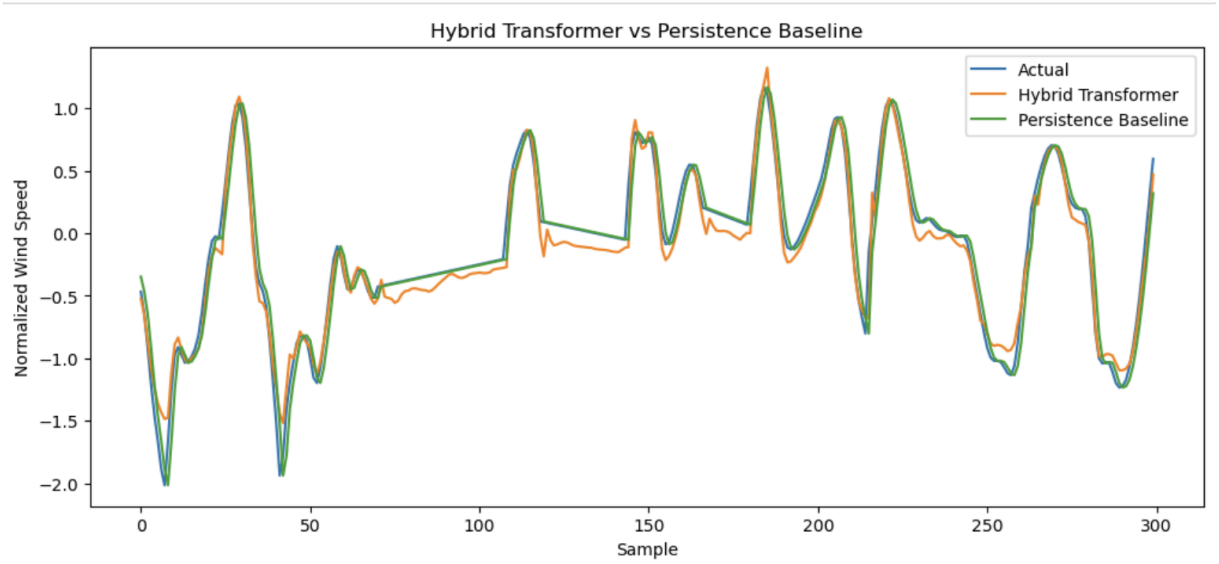
In this step, we performed learning rate optimization, which is a vital step while training deep learning models as the learning rate is a parameter that affects how much the model's weights are updated during gradient descent. We have used different learning rates (0.0001, 0.0005, 0.001, 0.005) while keeping other parameters constant (window size = 128, batch size = 64, number of epochs = 10) to observe the impact of learning rate on our model. The low learning rate (0.0001) resulted in slow convergence with smooth curves but increased computational time. However, learning rates between 0.0005 and 0.001 provided optimal results in terms of faster convergence as well as computational efficiency while achieving the minimum Mean Squared Error (MSE). On the other hand, a high learning rate (0.005) was seen to cause instability in training, characterized by varying loss values, the possibility of overshooting the optimal solution, and divergence. This, in turn, highlights the importance of the learning rate to play a vital role in achieving convergence since a small learning rate may cause underfitting, while a large learning rate may cause overfitting and divergence. Although the use of an adaptive optimizer such as Adam can be used to achieve stability in the model, it is important to consider the learning rate value. This can be achieved by employing learning rate scheduling. The optimal value for the learning rate was found to be 0.001, which ensures convergence with minimal errors.

Comparison with Persistence Forecast Baseline

As the next step in the validation of the hybrid model's effectiveness, we evaluated the effectiveness of the hybrid model by comparing it with a persistence forecast baseline, a widely used naive benchmark in time series forecasting. The persistence model assumes that the future value is equal to the most recent observed value, mathematically represented as $\hat{y}_t = y_{t-1}$. This is important because it provides a baseline for the model's performance, helps in determining whether the model is indeed learning from the patterns, and avoids overestimation of the model's performance. The persistence model is very simple, has low computation requirements, and performs well when there is high short-term autocorrelation in the data or when the data is stable. However, there are several drawbacks to this model because it does not perform well in the presence of trends, seasonality, sudden fluctuations, or volatility in the data, and it does not have the capability for learning from past patterns. On the contrary, the hybrid model uses sequence modeling for temporal dependencies and neural networks for non-linear relationships. Therefore, it has outperformed the persistence model in terms of having a lower Mean Squared Error (MSE), better adaptability in changing patterns in the data, and better forecasting for longer horizons. Thus, outperforming the persistence model is a measure of the hybrid model's capability for learning from the patterns.

Visualization – Hybrid vs Baseline vs Actual

We also visualized the performance of the model by plotting the actual values (ground truth), the predictions made by the hybrid model, and the predictions made by the persistence baseline model. This will give us a better idea about the performance of the model. By observing the above figure, we can clearly see that the hybrid model follows the actual values. This means the hybrid model has performed well by considering the peaks and dips in the values and adapting itself to the dynamic changes in the values. However, the persistence model has a lagging effect and does not consider the spikes and dips in the values. Instead, the values predicted by the persistence model seem to be shifted. Moreover, visualization is also an essential factor in detecting any problems such as underfitting and overfitting, understanding model behavior, and interpreting results. The findings from this study show that hyperparameter tuning, particularly optimization of the learning rate, has a significant effect on performance, and the hybrid model performs better than the baseline, even for short-term variations and long-term patterns. As for the suggestions for improvement in the future, it is recommended to use methods such as adjusting the learning rate using 'ReduceLROnPlateau,' preventing overfitting via 'early stopping,' and exploring other models such as 'LSTM' and 'Transformer,' larger data sets, and more exogenous variables.



Ethics Discussion

Reproducibility challenges

This project showcases multiple common reproducibility problems that exist in data science work. The paper delivers an excellent methodological explanation yet the research team needed to perform extensive debugging work together with interpretation tasks in order to create a functional system based on the published methodology. The research faced three main obstacles which included missing preprocessing information, issues with software version compatibility, and unclear connections between public code and the specific experimental procedures described in the paper. The project guidelines explicitly state that reproducibility work requires all elements which include missing hyperparameters and undefined preprocessing steps together with version conflicts and random seed issues and hardware differences because these factors prevent precise numerical duplication even when researchers comprehend the fundamental method.

Resource considerations

The ethical question of resource access creates problems for reproducing results. The guidelines state that researchers should not consider their work unsuccessful when they cannot replicate original results because of resource limitations which exist between different student and researcher groups. The reproduction work required adaptations to free notebook environments instead of using specialized training facilities. The project became easier to use

because of this change, but it created new challenges which required extra work to establish successful code operations. The process of scientific verification becomes more difficult when reproducibility requires access to specific computing resources and software configurations and secret research methodologies.

Scientific integrity implications

Scientists need to produce understandable results which other researchers can replicate because their work depends on their ability to generate trustworthy results. The project instructions frame reproducibility as an ethical issue tied to scientific integrity, transparency, access, and verification. The research demonstration showed that researchers face challenges in replicating results from a study which uses public datasets and public code because the paper uses a current publication. The paper remains valid because the situation does not invalidate it, yet the case demonstrates the importance of complete documentation. Future researchers will find it difficult to verify or extend the work because they lack access to information about the preprocessing steps and model configurations and the testing environment.

Recommendations

Best practices

The work can achieve better reproducibility through the implementation of several recommended best practices. First, papers should provide a complete description of preprocessing, including how missing values were handled, whether spikes were clipped, and how normalization was applied. Second, code should be published in a form that matches the experiments described in the paper as closely as possible, with clear instructions for execution. Third, environment files with exact package versions should be included so that later users can recreate the software stack more accurately. Fourth, random seeds, train-test splits, and stopping criteria should be explicitly documented.

Process improvements

For future reproduction attempts, the process can be improved by starting with smaller verified components before running the full pipeline, exactly as recommended in the project guidelines. It would also be helpful to compare multiple implementations of the same model, keep a detailed debugging log, and document all deviations from the original paper. In this specific case, a better future reproduction would align the public code, preprocessing, and target variable more closely with the exact wind power experiment reported in the paper. That would make the comparison more direct and might reduce the discrepancy between reproduced and published metrics.

Overall, this project shows that reproducibility should not be judged only by whether the final numbers match exactly. A careful attempt that documents methods, obstacles, discrepancies, and ethical implications still provides valuable scientific insight. The reproduction here supports the general effectiveness of the hybrid architecture while also demonstrating the practical difficulties of reproducing modern deep learning papers exactly.